Change Data Capture (CDC)

Debezium, Kafka and the dual-write problem

Watch the 3-min video

In one line: writing to two systems from the app (dual write) can silently leave them out of sync; with CDC the app writes only to the database, and a tool like Debezium streams every committed change from the database log, in commit order, to Kafka and on to search, caches and warehouses.

1 The bug: a price that never updates

Your product service saves a price change to Postgres, then updates Elasticsearch so search shows the new price. One day the Elasticsearch call **times out after the database commit**. Now search shows the old price, forever. Nothing retries it, and nobody notices for weeks.

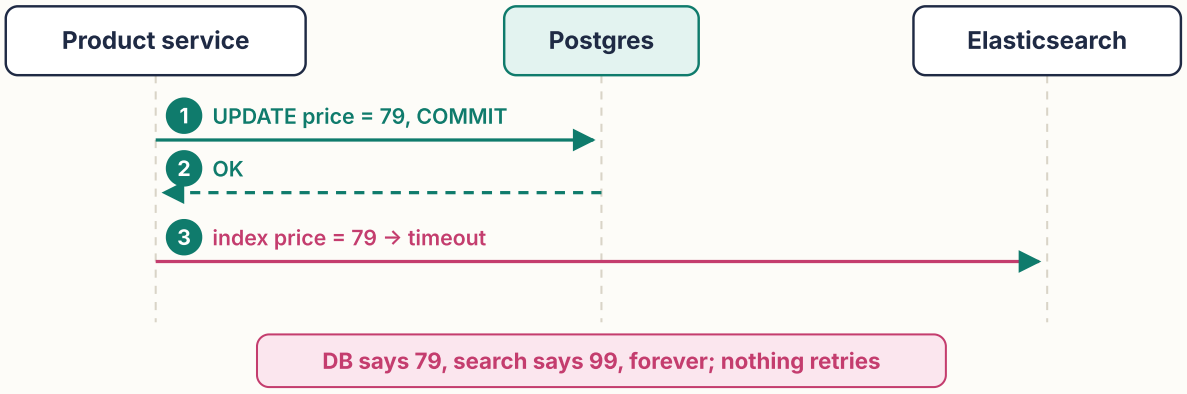


Figure 1. Step 1 succeeded and cannot be undone. Step 3 failed. The two systems now disagree, with no record of it.

2 The dual-write problem

When an application writes to **two systems**, there is no transaction across them. Either write can fail after the other succeeds. Retrying in the app does not fully fix it: the process can crash between the two writes.

And with **concurrent updates**, the two systems can apply them in **different orders**, ending up with different final values, with no error anywhere.

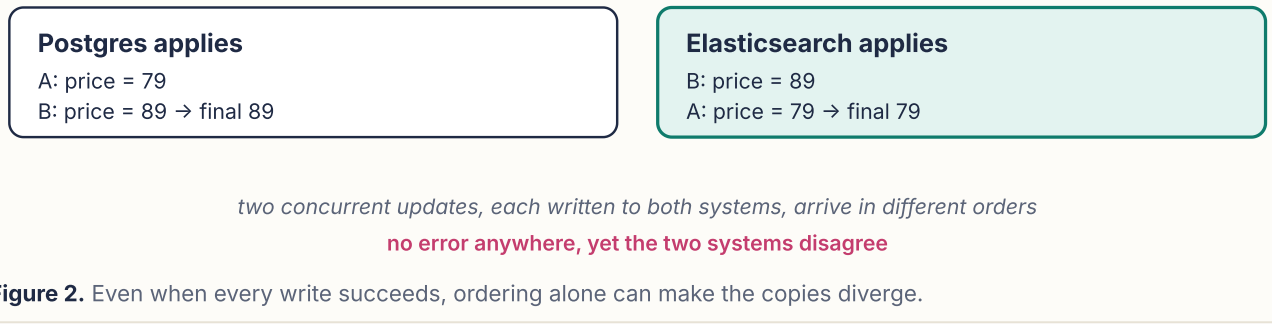


Figure 2. Even when every write succeeds, ordering alone can make the copies diverge.

3 Change data capture: read the database log

CDC takes a different path. The app writes **only to the database**, the single source of truth. Every database already records each committed change in its log: the **write-ahead log (WAL)** in Postgres, the **binlog** in MySQL. CDC reads that log and turns every insert, update and delete into an **event**, in commit order. Only committed changes appear; rolled-back ones never do.

The most popular tool is **Debezium**, which runs on **Kafka Connect**. For Postgres it uses **logical replication** with a **replication slot**. It first takes a **snapshot** of existing rows, then streams every new change. Each event carries the row **before** and **after** the change, the operation type, and its position in the log:

```
{ "op": "u",                        // c = insert, u = update, d = delete, r = snapshot read
  "before": { "id": 42, "price": 99 },
  "after":  { "id": 42, "price": 79 },
  "source": { "table": "products", "lsn": 23854104 },
  "ts_ms": 1767225600000 }
```

4 The pipeline, end to end

Debezium publishes product changes to a Kafka topic, **keyed by product ID**. Kafka keeps order within a partition and the key picks the partition, so all changes to one product stay in order. An **Elasticsearch sink connector** consumes the topic and **upserts** each document. If Elasticsearch is down for an hour, events wait in Kafka, and indexing resumes exactly where it stopped.

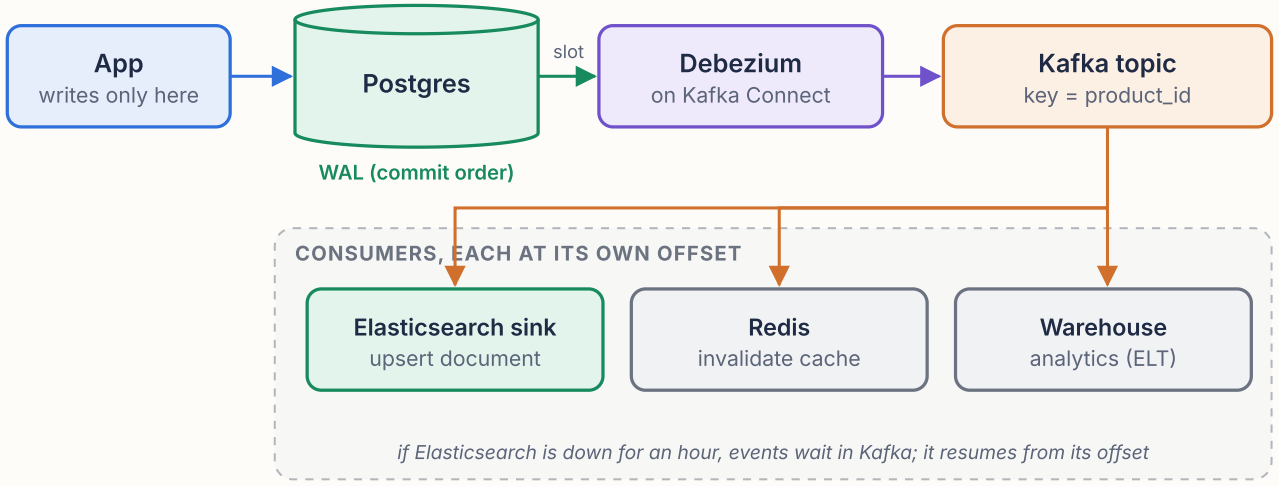


Figure 3. One write path, many readers. Each consumer tracks its own offset, so it can stop, fall behind and catch up independently.

The same stream can feed many consumers: **cache invalidation** in Redis, the **data warehouse** (the CDC feed from the last lesson), analytics, and other services.

When you need to publish **business events**, not just row changes, use the **outbox pattern**: in the **same transaction** as the business change, insert an event row into an `outbox` table, and let Debezium publish it.

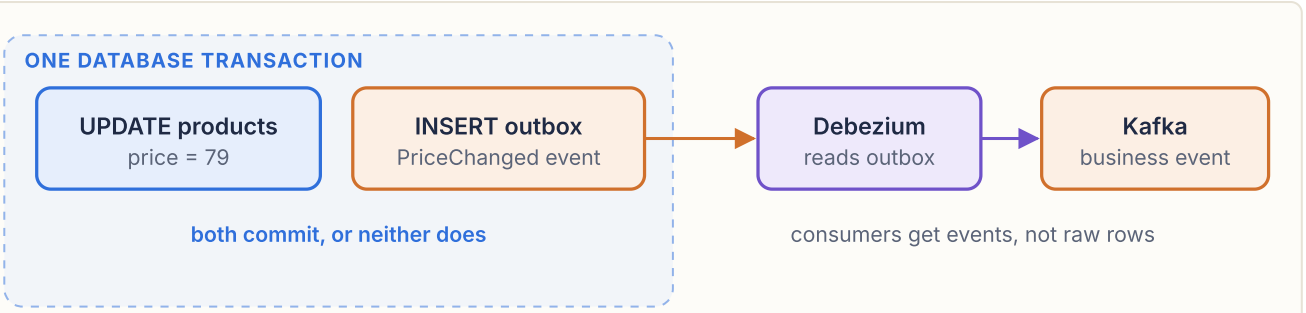


Figure 4. The outbox makes the event part of the same atomic commit, so the event exists if and only if the change does.

WORKED EXAMPLE: ONE-HOUR SEARCH OUTAGE

The catalog sees **200 price changes per second**. Elasticsearch is down for **1 hour**: 200 × 3,600 = **720,000 events** wait in Kafka (at ~1 KB each, under 1 GB). When it returns, the sink consumes at, say, 5,000 events/s and catches up in about **2.5 minutes**. With dual writes, those 720,000 updates would simply have been lost.

5 Watch out for

- **At-least-once delivery.** After a crash, some events are delivered again, so consumers must be **idempotent**: for example, upsert with a version and ignore older versions.
- **Replication slot growth.** A slot keeps the WAL until it is consumed. A stuck or deleted connector can **fill the database disk**. Monitor slot lag and alert on it.
- **Schema changes** flow into every consumer. Plan them (a schema registry helps) and make consumers tolerant of new fields.

COMMON MISTAKES

- "Write to the DB, then publish to Kafka" in app code: that is still a dual write.
- Non-idempotent consumers, so a replayed event double-counts.
- Keying the topic randomly, so one product's changes arrive out of order.
- No alert on replication-slot lag until the primary's disk is full.

6 Key terms

Dual write: app writes two systems, no shared transaction

CDC: stream committed changes from the DB log

WAL / binlog: Postgres / MySQL change log

Debezium: CDC connectors on Kafka Connect

Replication slot: keeps WAL until a consumer reads it

Snapshot: initial copy of existing rows

Sink connector: writes a topic into a target system

Outbox: event row in the same transaction

Idempotent: safe to apply twice

7 Interview questions

Q1. How does CDC avoid dual writes? BASIC

The app writes only to the database. CDC reads committed changes from the database log and publishes them in commit order to Kafka, from where search, caches and warehouses are updated. A consumer that is down resumes from its offset, so nothing is lost.

Q2. What is the dual-write problem? BASIC

Writing to two systems with no shared transaction. One write can fail after the other succeeds, or concurrent updates can land in different orders, leaving the systems permanently inconsistent.

Q3. Why key the Kafka topic by product ID? BASIC

Kafka orders messages only within a partition, and the key chooses the partition. Same key, same partition, so all changes to one product are consumed in commit order.

Q4. Elasticsearch was down for an hour. What happens with CDC? SCENARIO

Events accumulate in Kafka (e.g. 720,000 at 200/s). When Elasticsearch returns, the sink resumes from its committed offset and catches up; idempotent upserts make any replays harmless.

Q5. You need an OrderPlaced event, not raw row changes. How? SCENARIO

Use the outbox pattern: in the same transaction as the order insert, insert an event row into an outbox table. Debezium streams the outbox, so the event is published if and only if the order committed.

Q6. The Postgres disk is filling up, and pg_wal is huge. Likely cause? SCENARIO

A replication slot whose consumer (the Debezium connector) is stuck or gone, so Postgres keeps all WAL since. Fix or drop the slot, and monitor slot lag with an alert.

Q7. CDC delivers at least once. How do consumers stay correct? TRICKY

Make them idempotent: upsert by key with a version or log position, and ignore an event older than what is stored. A replayed event then changes nothing.

Q8. Why not just retry the Elasticsearch write in the app? TRICKY

The app can crash between the commit and the retry, and retries can still reorder concurrent updates. CDC derives everything from the committed log, which is durable and ordered.

Next: SD-E-032 · Distributed SQL Explained backendroadmap.com